

NOM	SAINSON
Prénom	Jocelyn
Date de naissance	27/02/2001

Copie à rendre

Bloc 3 – Développement d’une solution digitale avec Java

Documents à compléter et à rendre

Lien git :

- Global : <https://github.com/jos34000/JO2024.git>
- Submodule API : <https://github.com/jos34000/api-jo2024.git>
- Submodule frontend : <https://github.com/jos34000/ui-jo2024.git>

Lien gestion de projet : [gestion de projet scrum](#)

- [1er lien](#) (nécessité de se créer un compte ou se connecter avec votre propre compte puis accéder au projet nommé "Dev - JO2024")
- [2ème lien](#) (vous pouvez utiliser votre propre compte puis aller dans "Espaces" / "Spaces" sur le menu de gauche puis "Plus d'espaces" / "More spaces" et sélectionner "Dev - JO2024". Il peut y avoir des erreurs ou un certain temps de chargement avant que ce ne soit accessible)

Lien application déployée : <https://jo2024.marcelyn.fr>

Lien documentation swagger API : <https://api-jo2024.marcelyn.fr/swagger-ui/index.html>

Lien coverage tests : <https://jos34000.github.io/JO2024/>

Dossier 1 : Spécifier une solution digitale

1. Énumérez toutes les fonctionnalités attendues par le client sous forme d'User Story (agilité)

Voir dans le projet "Dev - JO2024" dans l'espace Jira. La plupart étant au statut Done, il faut aller dans l'onglet "List" puis bouton "Filter" et cliquer sur "Done work items" ou aller dans l'onglet "All work".

2. Quels sont les éléments que vous allez sécuriser ? Comment allez-vous procéder ?

Le client insiste particulièrement sur la sécurité des comptes utilisateurs. Ma stratégie de sécurisation couvre l'ensemble de la chaîne, de l'authentification jusqu'à la génération des e-billets.

1. Authentification et gestion des comptes utilisateurs

Menaces identifiées :

- Attaques par force brute
- Usurpation d'identité
- Vol de session

Solutions techniques :

- Politique de mot de passe forte : minimum 8 caractères incluant majuscules, minuscules, chiffres et caractères spéciaux
- Hashage sécurisé : utilisation de BCrypt avec salt pour le stockage des mots de passe (jamais en clair)
- Limitation des tentatives : blocage temporaire du compte après 5 tentatives échouées
- Authentification à deux facteurs (2FA) : envoi d'un code par email lors de la connexion depuis un nouvel appareil
- Gestion des sessions : tokens JWT avec expiration (durée limitée à 2h) et refresh token
- Déconnexion automatique : après 30 minutes d'inactivité

Implémentation :

- Spring Security + JWT (SecurityFilterChain)
- BCryptPasswordEncoder pour le hashage

2. Sécurisation des clés

Menaces identifiées :

- Falsification de billets
- Génération frauduleuse de QR codes
- Accès non autorisé aux clés

Solutions techniques :

- Clés (1 / 2 et finale) : stockée côté serveur uniquement, jamais transmise au client
- Stockage : clés chiffrées en base de données (AES-256)
- Validation : algorithme de vérification lors du scan permettant de déchiffrer et valider l'authenticité

Implémentation :

- java.security.SecureRandom pour génération
- MessageDigest SHA-256 pour hashage
- Lib ZXing pour génération QR Code sécurisé

3. Protection des données sensibles (RGPD)

Menaces identifiées :

- Fuite de données personnelles
- Non-conformité RGPD

Solutions techniques :

- Chiffrement en base : colonnes sensibles chiffrées avec AES-256
- Minimisation des données : collecte uniquement des informations nécessaires
- Droits utilisateurs : possibilité de consulter, modifier, supprimer ses données
- Logs d'accès : traçabilité des accès aux données sensibles
- Politique de conservation : suppression automatique des données après l'événement + délai légal

Implémentation :

- Respect des principes RGPD
- Consentement explicite lors de la création de compte

4. Sécurisation des accès et autorisations

Menaces identifiées :

- Élévation de privilèges
- Accès non autorisé à l'espace admin
- Injection de code

Solutions techniques :

- Contrôle d'accès basé sur les rôles (Role Base Access Control) : USER / ADMIN / STAFF avec privilèges distincts

- Compte administrateur / équipe : impossible à créer via l'application, fournit directement en base
- Séparation des endpoints : routes /admin/* /staff/* protégées avec vérification du rôle
- Validation des entrées : sanitization systématique côté backend
- Protection CSRF : tokens anti-CSRF pour toutes les opérations sensibles

Implémentation :

- Spring Security avec @PreAuthorize
- Enum pour les rôles (ROLE_USER, ROLE_ADMIN, ROLE_STAFF)
- CSRF protection activée
- Input validation avec @Valid et annotations

5. Sécurisation du processus de paiement

Menaces identifiées :

- Manipulation du montant
- Double paiement
- Injection dans le panier

Solutions techniques :

- Validation côté serveur : re calcul systématique du montant total
- Idempotence : génération d'un identifiant unique de transaction pour éviter les doublons
- États de transaction : gestion stricte (PENDING, VALIDATED, FAILED)
- Protection du panier : validation de la disponibilité des offres avant paiement

Implémentation :

- Vérification serveur des prix
- Transaction db avec ACID
- State pattern pour gestion des états

6. Infrastructure et communication

Menaces identifiées :

- Man-in-the-middle
- Injection SQL
- Attaques XSS/CSRF
- DDoS

Solutions techniques :

- HTTPS obligatoire : certificat SSL/TLS pour toutes les communications
- Protection injection SQL : utilisation exclusive de JPA
- Headers de sécurité :
 - Content-Security-Policy
 - X-Content-Type-Options: nosniff
 - X-Frame-Options: DENY
 - Strict-Transport-Security (HSTS)
- Rate limiting : limitation du nombre de requêtes par IP
- CORS configuré : whitelist des origines autorisées

- Logs de sécurité : enregistrement des tentatives suspectes

Implémentation :

- Spring Security headers configuration
- JPA/Hibernate pour requêtes préparées
- Redis pour rate limiting
- Déploiement avec HTTPS

7. Sécurisation de l'espace administrateur

Menaces identifiées :

- Accès non autorisé
- Manipulation des offres
- Exposition des statistiques sensibles

Solutions techniques :

- Authentification renforcée : 2FA obligatoire pour les admins
- Audit trail : logs de toutes les actions admin (création/modification/suppression d'offres)
- Validation des opérations CRUD : vérification systématique des droits
- Interface dédiée : séparation physique des routes admin

Implémentation :

- `@PreAuthorize("hasRole('ADMIN')")`
- `AuditingEntityListener` pour traçabilité

Cette stratégie de sécurisation multi couche garantit la protection du système à tous les niveaux. L'accent particulier mis sur l'authentification répond aux exigences du client, tandis que le système de double clé assure l'authenticité des e-billets.

3. Évoquez les choix techniques que vous avez choisis concernant votre application et justifiez-les (tout en faisant référence au besoin client)

Le choix de la stack technique s'appuie sur trois contraintes imposées par le contexte des JO : la performance (millions de requêtes quotidiennes attendues), la sécurité (exigence forte du client) et la fiabilité (événement international critique). Les choix technologiques répondent précisément aux contraintes fonctionnelles et techniques du projet.

1. Architecture globale : Architecture 3-tiers

Choix technique : Architecture client-serveur en 3 couches (Front / API / Data)

- Séparation des responsabilités : Frontend (Next.js), Backend (Spring Boot), Base de données (PostgreSQL) sont découplés
- Scalabilité horizontale : possibilité d'ajouter des instances backend pour gérer les pics de charge lors des JO
- Maintenance facilitée : chaque couche peut évoluer indépendamment
- Sécurité renforcée : le backend fait office de garde-fou, aucun accès direct à la base depuis le client

Ceci répond directement à l'exigence de gérer des millions de requêtes par jour tout en garantissant la sécurité maximale des données (clés, informations utilisateurs).

2. Frontend : React.js avec Next.js

Choix technique :

- React.js :
 - Navigation fluide sans rechargement
 - Composants réutilisables
 - Gestion d'état optimisée
 - Large écosystème
 - Performances optimales même avec forte charge
- Next.js:
 - Hot Module Replacement (HMR) instantané
 - Build optimisé
 - Temps de démarrage minimal
 - Configuration simplifiée

Qu'il s'agisse de besoin fonctionnels ou de développement cette stack répond à ces exigences :

- Respecte l'obligation d'une application dynamique sans rechargement de page
- Interface réactive pour affichage des offres, gestion du panier en temps réel
- Expérience utilisateur fluide indispensable pour un événement grand public
- Performance optimale pour supporter le trafic massif attendu

3. Backend : Java Spring Boot

Pourquoi ?

- Framework enterprise-grade
- Spring Security
- Spring Data JPA
- Architecture RESTful
- Injection de dépendances : code maintenable et testable
- Écosystème riche
- Performance

Cela répond à :

- Spring Security = solution idéale pour l'exigence de sécurité maximale
- Capable de gérer la charge quotidienne grâce à sa robustesse
- Spring Data JPA = requêtes optimisées pour gérer les offres, utilisateurs, clés, transactions

4. Base de données : PostgreSQL

Qu'est ce que ça apporte ?

- SGBD relationnel robuste
- ACID garanti
- Performances élevées
- Types de données avancés : JSON, UUID natifs
- Sécurité : chiffrement des données au repos et en transit, gestion fine des permissions
- Scalabilité : réplication master-slave possible pour répartir la charge en lecture
- Open source

Lien avec le besoin client :

- Respecte la contrainte obligatoire de base relationnelle
- Intégrité référentielle : crucial pour lier utilisateurs ↔ clés ↔ billets ↔ offres
- Transactions ACID : garantit la cohérence lors du processus de paiement
- Type UUID : stockage optimisé des clés générées
- Capacité de gérer le volume important de données (millions de visiteurs potentiels)

5. Sécurité : Spring Security + JWT + BCrypt

Spring Security (framework de sécurité) :

- Protection multicouche : filtres HTTP, contrôle d'accès aux endpoints
- RBAC natif : gestion des rôles USER/ADMIN/STAFF
- Protection CSRF/XSS : activée par défaut
- Configuration déclarative : @PreAuthorize, SecurityFilterChain

JWT (tokens d'authentification stateless) :

- Stateless : pas de stockage serveur de sessions, scalabilité optimale
- Sécurisé : signature cryptographique (HS256 ou RS256)
- Payload personnalisable : inclusion du rôle, ID utilisateur, expiration

BCrypt (algorithme de hashage de mots de passe) :

- Salt automatique : protection contre rainbow tables
- Coût paramétrable : adaptation à la puissance de calcul disponible
- One-way hashing : impossible de retrouver le mot de passe original

Avec ces choix, on répond aux besoins client suivants:

- Exigence forte de sécurité
- JWT = solution moderne pour authentification forte sans surcharge serveur
- BCrypt = protection des comptes utilisateurs (insistance client sur ce point)
- Spring Security = gestion sécurisée

6. Génération de QR Code : ZXing

- Standard de l'industrie
- Multi-formats : QR Code, DataMatrix, etc.
- Personnalisation : taille, niveau de correction d'erreur
- Génération serveur : sécurité maximale, génération côté backend uniquement
- Format de sortie flexible : PNG, SVG
- Performance : génération rapide même en volume

Qu'est-ce que ça apporte ?

- Génération du e-billet sécurisé à partir de la clé finale (concaténation hashé clé1 + clé2)
- QR Code = solution anti-fraude, chaque billet est unique et vérifiable
- Niveau de correction d'erreur Q ou H pour lecture fiable lors du scan à l'entrée des JO
- Génération backend = impossibilité de falsification côté client

7. Déploiement : Docker + Cloudflare Tunnel + Cloudflare

- Docker : conteneurisation de l'application
- Cloudflare Tunnel : exposition sécurisée sans ouverture de ports
- Cloudflare : CDN, DNS, protection DDoS, SSL/TLS

Docker :

- Isolation : conteneurs frontend, backend, PostgreSQL séparés
- Reproductibilité : `docker-compose.yml` garantit un environnement identique dev/prod
- Portabilité : déploiement facilité sur tout serveur supportant Docker
- Gestion des ressources : limites CPU/RAM par conteneur

Cloudflare Tunnel :

- Sécurité renforcée : pas d'ouverture de ports 80/443 sur le serveur
- Protection DDoS : filtrage du trafic par Cloudflare avant d'atteindre le serveur
- Connexion chiffrée : tunnel TLS entre serveur et Cloudflare
- IP serveur masquée : protection contre attaques directes

Cloudflare :

- CDN global : cache statique (assets React) sur edge servers mondiaux = latence minimale
- SSL/TLS automatique : certificats gérés par Cloudflare, HTTPS obligatoire
- WAF (Web Application Firewall) : protection contre OWASP Top 10 (injections, XSS, etc.)
- Rate limiting : protection contre abus et scraping
- Analytics temps réel : monitoring du trafic

Tout cela permet :

- Scalabilité : Docker swarm facilite l'ajout d'instances backend si millions d'appels simultanés
- Sécurité maximale : Cloudflare WAF + Tunnel = protection multicouche contre attaques
- Performance globale : CDN Cloudflare = temps de chargement optimisé pour visiteurs internationaux
- Haute disponibilité : Cloudflare garantit l'uptime, crucial pour événement ponctuel
- Protection DDoS : essentiel face à la visibilité d'un événement comme les JO
- HTTPS garantit : sécurité des communications (mots de passe, clés, paiement)

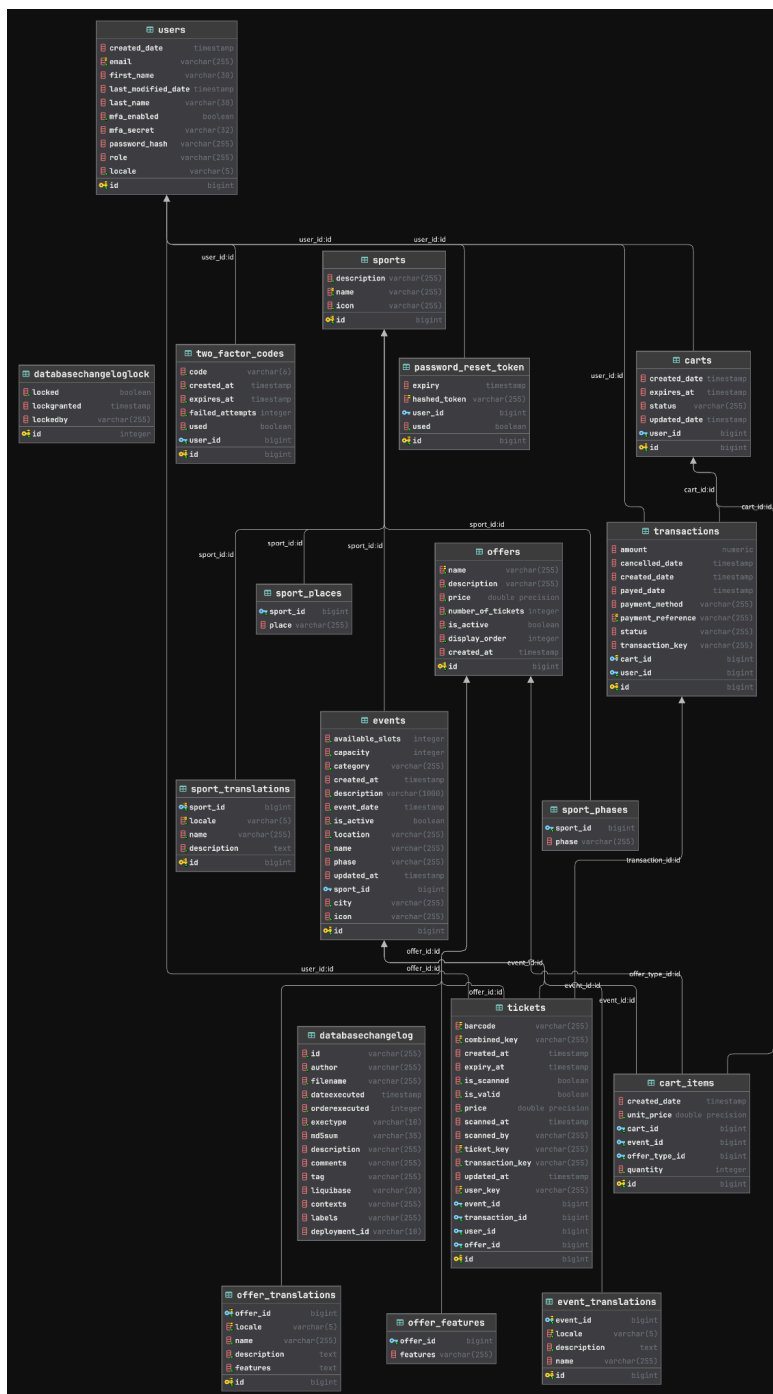
Dossier 2 : Développement de la solution

2.1 Développez la solution demandée par le client, pour ce faire, vous devrez également :

-Produire le MCD

Liens des images / graphiques :

- PNG : [Google Drive](#) / [Github](#)
- Drawio : [Google Drive](#) / [Github](#)
- UML : [Google Drive](#) / [Github](#)



-Effectuer une gestion de projet

- o Fournir l'outil de gestion de projet que vous avez utilisé suivant la méthode KANBAN
Lien fourni plus haut.

-Produire une documentation technique

- o Cette documentation devra aborder la sécurité, mais également, des évolutions futures de votre application

Lien du document :

- PDF : [Google Drive](#) / [Github](#)
- Markdown : [Google Drive](#) / [Github](#)

-Produire un manuel d'utilisation

- o Conception d'une documentation permettant à l'utilisateur d'utiliser votre application, on peut voir cela comme un « tuto »

Lien du document :

- PDF : [Google Drive](#) / [Github](#)
- Markdown : [Github](#)

-Le client vous impose :

- o D'effectuer un back-end -> OK
- o D'effectuer une application dynamique avec du JavaScript (le JavaScript permet de contacter votre back-end sans rechargement de page) -> OK
- o Mettre en place des tests et produire un rapport détaillant le pourcentage de code total couvert par les tests -> Lien du test coverage jacoco : <https://jos34000.github.io/JO2024/>
- o Déploiement en ligne -> OK